Numerical Analysis: Final Exam

(**50 marks**, only the 4 best questions count)

Solutions

Urbain Vaes

December 2024

Question 1 (Floating point arithmetic, **10 marks**)**.**

1. **(T/F)** Let $(\bullet)_2$ denote base 2 representation. It holds that
$$3 \times (0.0101)_2 = (0.1111)_2.$$
2. **(T/F)** Does the following equality hold? Explain your reasoning.
$$(0.\overline{011})_2 = \frac{3}{4}.$$
3. **(T/F)** In Julia, `Float64(x) == Float32(x)` is `true` if `x` is a rational number.
4. **(T/F)** The value of the machine epsilon for the double precision format is the same in Julia and Python.
5. **(T/F)** The spacing (in absolute value) between successive double-precision (`Float64`) floating point numbers is equal to the machine epsilon.
6. **(T/F)** All the natural numbers can be represented exactly in the double precision floating point format `Float64`.
7. **(T/F)** Machine addition in the `Float64` format is associative but not commutative.
8. **(T/F)** In Julia, let `f(x) = (x == x/100.0) ? x : f(x/100.0)`. Then `f(a)` returns `0.0` for all finite number `a` representable in the `Float64` format.
9. (**1 mark**) In Julia `exp(eps()) == 1 + eps()` evaluates to `true`. Explain briefly why.
10. (**1 mark**) In Julia `sqrt(1 + eps()) == 1 + eps()` evaluates to `false`. Explain briefly why.

Solution. Below, $\varepsilon = 2^{-52}$ denotes the machine epsilon for `Float64`.

1. **True**: $3 \times (0.0101)_2 = 3 \times \frac{5}{16} = \frac{15}{16} = (0.1111)_2$.
2. **False**: if $x = (0.\overline{011})_2$, then $2^3 x = (11)_2 + x = 3 + x$, so $x = \frac{3}{7} \neq \frac{3}{4}$.

3. **False**: for example, $\frac{1}{3}$ is not representable exactly in either format, and its approximations in single and double precision differ, so `Float64(1/3) == Float32(1/3)` evaluates to `false`.
4. **True**: both languages use the IEEE 754 double precision format, for which the machine epsilon is $2^{-52} \approx 2.2 \times 10^{-16}$.
5. **False**: the spacing between successive `Float64` numbers in $[2^e, 2^{e+1})$ is $2^e\varepsilon$, so it depends on the magnitude of the numbers.
6. **False**: only finitely many numbers are representable. For example, $2^{53}+1$ is not representable in `Float64`.
7. **False**: it is the opposite: machine addition is commutative but not associative. For example, $(1 \mathbin{\widehat{+}} \varepsilon/2) \mathbin{\widehat{+}} \varepsilon/2 = 1$, whereas $1 \mathbin{\widehat{+}} (\varepsilon/2 \mathbin{\widehat{+}} \varepsilon/2) = 1 + \varepsilon$.
8. **True**: the condition `x == x/100.0` holds only if $x = 0$ (or $x = \pm\infty$, which is excluded). Dividing a finite number repeatedly by 100, we obtain a number that underflows to 0 after finitely many steps, at which point the recursion stops. (Strictly speaking, if `a` is negative, the function returns `-0.0`, which compares equal to `0.0`.)
9. By Taylor's theorem, $\exp(\varepsilon) = 1 + \varepsilon + \frac{\varepsilon^2}{2} + \cdots$, and $\frac{\varepsilon^2}{2}$ is negligible compared with the spacing ε between $1+\varepsilon$ and the next floating point number. Since $\exp(\varepsilon)$ is very close to the floating point number $1+\varepsilon$, it is rounded to this number.
10. The number $1+\varepsilon$ is representable exactly, and $\sqrt{1+\varepsilon} = 1 + \frac{\varepsilon}{2} - \frac{\varepsilon^2}{8} + \cdots$. This is slightly smaller than $1 + \frac{\varepsilon}{2}$, the midpoint between 1 and $1+\varepsilon$, so the result is rounded to $1 \neq 1+\varepsilon$.

△

Question 2 (Interpolation, **10 marks**).

1. **(T/F)** The only polynomial p of degree at most 3 such that $p(-1) = p(0) = p(1) = 1$ is the constant polynomial $p(x) = 1$.
2. **(T/F)** In polynomial interpolation, using Chebyshev nodes can help reduce the interpolation error compared to using equidistant nodes.
3. **(2 marks)** Let $S(n) = \sum_{i=1}^{n} i$. Given that $S(n)$ is a quadratic polynomial, calculate the expression of $S(n)$ by interpolation. Include the details of your calculation.
4. **(T/F)** Given $x_0 < x_1 < x_2$ and $y_0, y_1, y_2 \in \mathbb{R}$, the unique polynomial passing through these data points is given by
$$p(x) = \frac{(x-x_1)(x-x_2)}{(x_0-x_1)(x_0-x_2)}y_0 + \frac{(x-x_0)(x-x_2)}{(x_1-x_0)(x_1-x_2)}y_1 + \frac{(x-x_0)(x-x_1)}{(x_2-x_0)(x_2-x_1)}y_2.$$
5. **(T/F)** Let $f(x) = \exp(5x)$, and for any $n \in \mathbb{N}$, let $f_n \in \mathcal{P}_n$ denote the polynomial interpolating f at $n+1$ equidistant points $-1 = x_0 < x_1 < \ldots < x_n = 1$. Then
$$\lim_{n\to\infty} \left(\max_{-1 \leqslant x \leqslant 1} \left| f(x) - f_n(x) \right| \right) = 0.$$
6. **(2 marks)** Given $x_0 < \ldots < x_n$ and $y_0, \ldots, y_n \in \mathbb{R}$, prove that the constant polynomial p that

minimizes the expression $\sum_{i=0}^{n} |y_i - p(x_i)|^2$ is given by

$$p(x) = \frac{1}{n+1} \sum_{i=0}^{n} y_i.$$

7. (2 marks) We wish to find a, b, c such that the function $f(x) := a\cos(x) + b\sin(x) + c$ interpolates the data points $(0,0), (1,1), (2,2)$. Complete on paper the following code for calculating a, b, c.

```
x = [0.0, 1.0, 2.0]
y = [0.0, 1.0, 2.0]
# Your code below
```

Solution.

1. **False**: for any $c \in \mathbf{R}$, the polynomial $p(x) = 1 + c(x+1)x(x-1)$ has degree at most 3 and satisfies the conditions. The interpolating polynomial is unique only in $\mathcal{P}_2$.
2. **True**: for Chebyshev nodes, the product $\prod_{i=0}^{n}(x - x_i)$ in the interpolation error formula is minimized in the supremum norm, and in particular the Runge phenomenon is avoided.
3. Since S is a quadratic polynomial, it coincides with its interpolant at three points, for example $S(0) = 0$, $S(1) = 1$ and $S(2) = 3$. Using the Lagrange form,

$$S(n) = 0 \times \frac{(n-1)(n-2)}{2} + 1 \times \frac{n(n-2)}{-1} + 3 \times \frac{n(n-1)}{2} = \frac{-2n^2 + 4n + 3n^2 - 3n}{2} = \frac{n(n+1)}{2}.$$

4. **True**: this is the Lagrange form of the interpolating polynomial, which is of degree at most 2 and satisfies $p(x_i) = y_i$ for $i \in \{0, 1, 2\}$.
5. **True**: for $x \in [-1, 1]$, the interpolation error satisfies

$$|f(x) - f_n(x)| = \left| \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n}(x - x_i) \right| \leqslant \frac{5^{n+1}\mathrm{e}^5}{(n+1)!} 2^{n+1} = \mathrm{e}^5 \frac{10^{n+1}}{(n+1)!} \xrightarrow[n\to\infty]{} 0.$$

6. Let $J(c) = \sum_{i=0}^{n} |y_i - c|^2$. This is a convex quadratic function of c, with $J''(c) = 2(n+1) > 0$, so its minimizer is the unique root of

$$J'(c) = -2\sum_{i=0}^{n}(y_i - c) = 2(n+1)c - 2\sum_{i=0}^{n} y_i,$$

that is $c = \frac{1}{n+1}\sum_{i=0}^{n} y_i$.

7. The interpolation conditions $a\cos(x_i) + b\sin(x_i) + c = y_i$ form a linear system for (a, b, c):

```
x = [0.0, 1.0, 2.0]
y = [0.0, 1.0, 2.0]
A = [cos.(x) sin.(x) ones(3)]
a, b, c = A \ y  # a = -1, b = 0.642..., c = 1
```

△

Question 3 (Integration, **10 marks**).

1. **(T/F)** The degree of precision of the following quadrature rule is 1:

$$\int_{-1}^{1} f(x)\,\mathrm{d}x \approx 2f(0).$$

2. **(T/F)** The closed Newton–Cotes rule with n points is exact for all linear polynomials.

3. **(T/F)** The degree of precision of the following quadrature rule is 5:

$$\int_{-1}^{1} f(x)\,\mathrm{d}x \approx 2f(0) + \frac{2}{3}f''(0) + \frac{2}{5}f^{(4)}(0).$$

4. **(T/F)** Suppose that $f \in C^\infty[a,b]$ and let $I_n[f]$ denote the approximate integral of f using the composite trapezium rule with n integration points. Then it holds that

$$\lim_{n\to\infty}\Big|I[f] - I_n[f]\Big| = 0, \qquad I[f] := \int_a^b f(x)\,\mathrm{d}x.$$

5. **(T/F)** Suppose that $f \in C^\infty[a,b]$ and let $I_n[f]$ denote the approximate integral of f using the composite trapezium rule with n integration points. Then it holds that

$$\lim_{n\to\infty} n^2\Big|I[f] - I_n[f]\Big| < \infty.$$

6. **(2 marks)** Calculate weights w_1, w_2 so that the degree of precision of the following rule is 1:

$$\int_{-1}^{1} f(x)\,\mathrm{d}x = w_1 f(0) + w_2 f(1).$$

7. **(2 marks)** Implement the composite trapezium rule with n points:

```
function I_approx(f, a, b, n)
    x = LinRange(a, b, n)
```

8. **(1 mark)** The following code implements the midpoint rule with n points, but there is an error. Spot and correct the error.

```
function I_approx(f, a, b, n)
    x = LinRange(a, b, n)
    h = (b - a) / n
    return h * sum(f, x .+ h/2)
end
```

Solution. **1. True**: the rule (midpoint rule) is exact for 1 and x, but not for x^2, since $2 \times 0 \neq \int_{-1}^{1} x^2\,\mathrm{d}x = \frac{2}{3}$.

2. **True**: the closed Newton–Cotes rule with $n \geqslant 2$ points is obtained by integrating the interpolating polynomial of degree $n-1$, so it is exact for polynomials of degree at most $n-1$, and in particular for linear polynomials.

3. **False**: for $f(x) = x^2$, the rule gives $\frac{2}{3} \times 2 = \frac{4}{3}$, whereas $\int_{-1}^{1} x^2 \, \mathrm{d}x = \frac{2}{3}$. The rule is exact for 1, x and x^3, but not x^2, so its degree of precision is 1. (A Taylor expansion shows that the correct coefficients are $\frac{1}{3}$ and $\frac{1}{60}$ instead of $\frac{2}{3}$ and $\frac{2}{5}$.)

4. **True**: the error of the composite trapezium rule is bounded by $\frac{(b-a)h^2}{12} \max_{[a,b]} |f''|$, with $h = \frac{b-a}{n-1}$.

5. **True**: by the same error bound, $n^2 \big| I[f] - I_n[f] \big|$ is bounded uniformly in n. (In fact, it converges to $\frac{(b-a)^2}{12} |f'(b) - f'(a)|$.)

6. Exactness for $f(x) = 1$ and $f(x) = x$ gives $w_1 + w_2 = 2$ and $w_2 = 0$, so $w_1 = 2$ and $w_2 = 0$. The degree of precision is 1, because the rule is not exact for x^2.

7.
```
function I_approx(f, a, b, n)
    x = LinRange(a, b, n)
    h = (b - a) / (n - 1)
    return h * (sum(f, x) - (f(a) + f(b)) / 2)
end
```

8. The points `x .+ h/2` are not the midpoints of the n cells: `LinRange(a, b, n)` has spacing $\frac{b-a}{n-1}$, not h, and the last point $b + \frac{h}{2}$ lies outside the interval. The midpoints are `LinRange(a + h/2, b - h/2, n)`:

```
function I_approx(f, a, b, n)
    h = (b - a) / n
    x = LinRange(a + h/2, b - h/2, n)
    return h * sum(f, x)
end
```

△

Question 4 (Iterative method for linear systems, **10 marks**)**.** Assume that $\mathsf{A} \in \mathbf{R}^{n \times n}$ is an *invertible* matrix and that $\boldsymbol{b} \in \mathbf{R}^n$. We wish to solve the linear system

$$\mathsf{A}\boldsymbol{x} = \boldsymbol{b}. \tag{1}$$

1. (**3 marks**) We first consider a basic iterative method where each iteration is of the form

$$\mathsf{M}\boldsymbol{x}_{k+1} = \mathsf{N}\boldsymbol{x}_k + \boldsymbol{b}. \tag{2}$$

Here $\mathsf{A} = \mathsf{M} - \mathsf{N}$ is a splitting of A such that M is nonsingular, and $\boldsymbol{x}_k \in \mathbf{R}^n$ denotes the k-th iterate of the numerical scheme. Let $\boldsymbol{e}_k := \boldsymbol{x}_k - \boldsymbol{x}_*$, where $\boldsymbol{x}_*$ is the exact solution to (1). Prove that the error satisfies

$$\forall k \in \mathbf{N}, \qquad \boldsymbol{e}_{k+1} = \mathsf{M}^{-1}\mathsf{N}\boldsymbol{e}_k.$$

2. **(T/F)** If $\|\mathsf{M}^{-1}\mathsf{N}\|_2 < 1$, then the iterative method (2) is convergent.

3. **(T/F)** The Gauss–Seidel iterative method is a particular case of (2).

4. (**3 marks**) Write down on paper a few iterations of the Jacobi method when

$$\mathsf{A} = \begin{pmatrix} 1 & 2 \\ 0 & 1 \end{pmatrix}, \qquad \boldsymbol{b} = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad \boldsymbol{x}_0 = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

   Is the method convergent?

5. (**2 marks**) Suppose that A is lower triangular. Implement the forward substitution algorithm for solving (1) in this case.

```
A = [1.0 0.0 0.0; 1.0 2.0 0.0; 1.0 2.0 3.0]
b = [1.0; 3.0; 6.0]
```

Solution. **1.** Since $\mathsf{A}\boldsymbol{x}_* = \boldsymbol{b}$, the exact solution satisfies $\mathsf{M}\boldsymbol{x}_* = \mathsf{N}\boldsymbol{x}_* + \boldsymbol{b}$. Subtracting this equation from (2), we obtain $\mathsf{M}\boldsymbol{e}_{k+1} = \mathsf{N}\boldsymbol{e}_k$, and so $\boldsymbol{e}_{k+1} = \mathsf{M}^{-1}\mathsf{N}\boldsymbol{e}_k$.

2. **True**: by the previous item, $\|\boldsymbol{e}_k\|_2 \leqslant \|\mathsf{M}^{-1}\mathsf{N}\|_2^k \|\boldsymbol{e}_0\|_2 \to 0$.

3. **True**: it corresponds to $\mathsf{M} = \mathsf{D} + \mathsf{L}$ and $\mathsf{N} = -\mathsf{U}$, where D, L and U are the diagonal, strictly lower triangular and strictly upper triangular parts of A.

4. For the Jacobi method, $\mathsf{M} = \mathsf{D} = \mathsf{I}$ and $\mathsf{N} = \mathsf{I} - \mathsf{A}$, so the iteration reads

$$\boldsymbol{x}_{k+1} = \begin{pmatrix} 0 & -2 \\ 0 & 0 \end{pmatrix} \boldsymbol{x}_k + \begin{pmatrix} 1 \\ 1 \end{pmatrix}.$$

Therefore

$$\boldsymbol{x}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \qquad \boldsymbol{x}_2 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}, \qquad \boldsymbol{x}_3 = \begin{pmatrix} -1 \\ 1 \end{pmatrix}, \qquad \dots$$

The method converges, and in fact $\boldsymbol{x}_k = \boldsymbol{x}_* = (-1, 1)^T$ for all $k \geqslant 2$: the iteration matrix $\mathsf{M}^{-1}\mathsf{N}$ is nilpotent, with $(\mathsf{M}^{-1}\mathsf{N})^2 = 0$, so its spectral radius is 0.

5.
```
function forward_subs(A, b)
    n = length(b)
    x = zeros(n)
    for i in 1:n
        s = b[i]
        for j in 1:i-1
            s -= A[i, j] * x[j]
        end
        x[i] = s / A[i, i]
    end
    return x
end
forward_subs(A, b)  # [1.0, 1.0, 1.0]
```

△

Question 5 (Nonlinear equations, **10 marks**)**.** We consider the following iterative method for cal-

culating $\sqrt[3]{2}$:

$$x_{k+1} = F(x_k), \qquad F(x) := \omega x + (1-\omega)\frac{2}{x^2}, \tag{3}$$

with $\omega \in [0,1]$ a fixed parameter.

1. (**2 marks**) Show that $x_* := \sqrt[3]{2}$ is a fixed point of the iteration (3).
2. (**4 marks**) Write down a Julia program based on the iteration (3) for calculating $\sqrt[3]{2}$. Use an appropriate stopping criterion that does not require to know the value of $\sqrt[3]{2}$.
3. **(T/F)** The iterative method (3) converges to $\sqrt[3]{2}$ for any $\omega \in [0,1]$.
4. **(T/F)** The secant method is usually faster than the Newton–Raphson method.
5. **(T/F)** The following iteration converges to $\sqrt[3]{2}$ for all initialization x_0:

$$x_{k+1} = x_k - \frac{x_k^3 - 2}{10}.$$

6. (**1 mark**) Illustrate a few iterations of the Newton–Raphson iteration for finding $\sqrt[3]{2}$ on the figure provided with the exam, where the function $f(x) = x^3 - 2$ is plotted.

Solution. **1.** Since $x_*^3 = 2$, it holds that $\frac{2}{x_*^2} = x_*$, and so $F(x_*) = \omega x_* + (1-\omega)x_* = x_*$.

2. We stop when two successive iterates are sufficiently close:

```
function cbrt2(w; x0 = 1.0, tol = 1e-12, maxiter = 1000)
    F(x) = w * x + (1 - w) * 2 / x^2
    x = x0
    for _ in 1:maxiter
        x_new = F(x)
        if abs(x_new - x) < tol
            return x_new
        end
        x = x_new
    end
    error("No convergence")
end
cbrt2(2/3)  # 1.2599210498948732
```

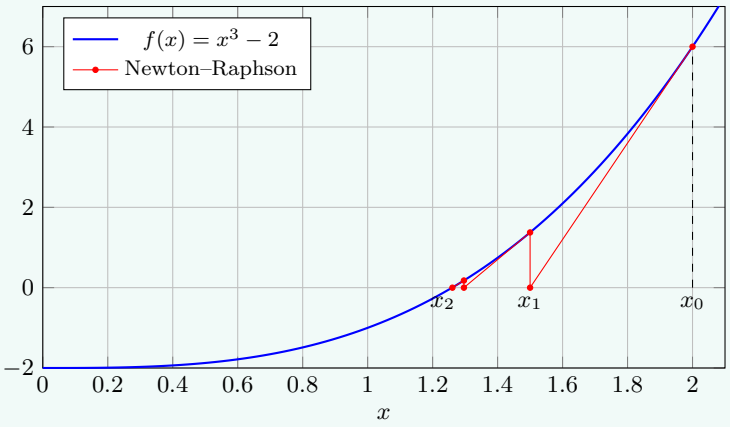
3. **False**: $F'(x) = \omega - 4(1-\omega)x^{-3}$, so $F'(x_*) = \omega - 2(1-\omega) = 3\omega - 2$. The fixed point is attracting if $|3\omega - 2| < 1$, i.e. $\omega \in (\frac{1}{3}, 1)$, and repelling if $\omega < \frac{1}{3}$. For example, for $\omega = 0$ the iteration $x_{k+1} = 2/x_k^2$ diverges from x_* unless $x_0 = x_*$. For $\omega = 1$, we have $F(x) = x$ and the iterates do not move. (The choice $\omega = \frac{2}{3}$, for which $F'(x_*) = 0$, corresponds to the Newton–Raphson method for $x^3 - 2$.)

4. **False**: the Newton–Raphson method converges quadratically, whereas the order of convergence of the secant method is $\frac{1+\sqrt{5}}{2} \approx 1.62$, so the Newton–Raphson method usually requires fewer iterations. (The secant method does not require the derivative, however, and so each iteration

may be cheaper.)

5. **False**: for large $|x_0|$ the iterates diverge. For example, $x_0 = 5$ gives $x_1 = 5 - 12.3 = -7.3$, $x_2 \approx 31.8$, and so on, with $|x_k| \to \infty$.

6. The iteration reads $x_{k+1} = x_k - \frac{x_k^3 - 2}{3x_k^2}$: x_{k+1} is the intersection with the x-axis of the tangent to the graph of f at $\big(x_k, f(x_k)\big)$. Starting from $x_0 = 2$, we obtain $x_1 = 1.5$, $x_2 \approx 1.2963$, $x_3 \approx 1.2609$, and $x_* \approx 1.2599$:



△